

Name : D.Mani Kumar Reddy

Reg No : 192325095

Course Code : ITA0515

Course : Computer Vision

Slot : B

Experiment 1: Read an Image and Convert it to Grayscale

Aim

To read an image using OpenCV and convert it into grayscale.

Algorithm

1. Start the program.
2. Import the OpenCV library.
3. Read the input image from the specified file path using `cv2.imread()`.
4. Verify that the image has been loaded successfully.
5. Convert the color image into a grayscale image using `cv2.cvtColor()` with the `COLOR_BGR2GRAY` conversion flag.
6. Store the grayscale image in a separate variable.
7. Display the original image in one window.
8. Display the grayscale image in another window.
9. Wait for the user to press any key.
10. Close all display windows and terminate the program.
11. .

Python Code

```
import cv2

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

cv2.imshow("Original Image", img)
cv2.imshow("Grayscale Image", gray)

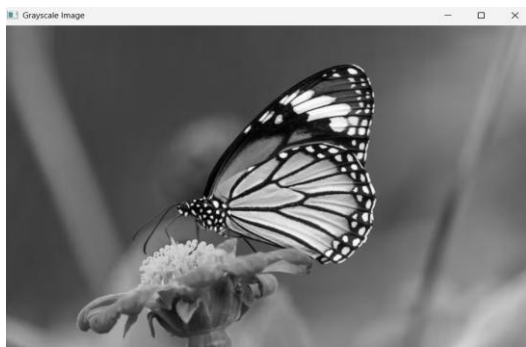
cv2.imwrite("gray_image.jpg", gray)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input Image:



Output Image:



Result:

The input image was successfully read and converted into a grayscale image.

Experiment 2: Read an Image and Apply Gaussian Blur

Aim

To read an image using OpenCV and apply Gaussian Blur to reduce noise and smooth the image.

Algorithm

1. Start the program.
2. Import the OpenCV library.
3. Read the input image from the specified file path.
4. Verify that the image is loaded successfully.
5. Choose a Gaussian kernel size (7×7).
6. Apply the Gaussian Blur function using `cv2.GaussianBlur()`.
7. Store the blurred image.

8. Display the original image.
9. Display the blurred image.
10. Wait for a key press and close all windows.

Python Code

```
import cv2

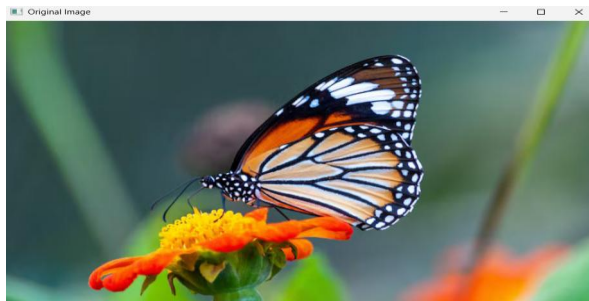
img = cv2.imread("image.jpg")

blur = cv2.GaussianBlur(img, (7, 7), 0)

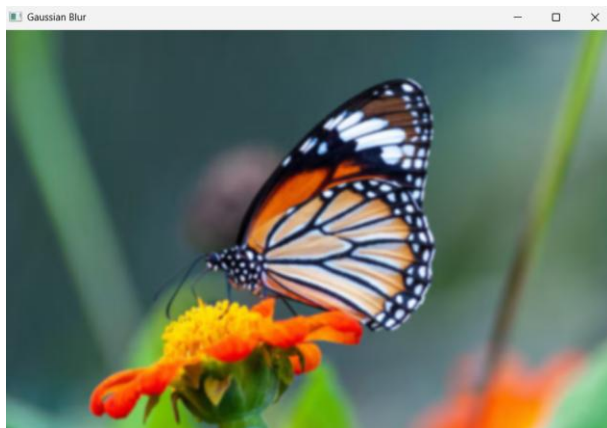
cv2.imshow("Original Image", img)
cv2.imshow("Gaussian Blur", blur)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input Image:



Output image:



Experiment 3: Read an Image and Detect Edges using Canny

Aim

To detect the edges present in an image using the Canny edge detection algorithm.

Algorithm

1. Start the program.
2. Import the OpenCV library.
3. Read the input image.
4. Convert the image into grayscale.
5. Specify the lower and upper threshold values.
6. Apply the Canny edge detection function.
7. Store the detected edge image.
8. Display the original image.
9. Display the edge-detected image.
10. Wait for a key press and close all windows.

Python Code

```
import cv2

img = cv2.imread(r"C:\Users\manik\OneDrive\Desktop\computer vision\image.jpg")

if img is None:
    print("Image not found!")
else:
    edges = cv2.Canny(img, 100, 200)

    cv2.imshow("Original Image", img)
    cv2.imshow("Canny Edge", edges)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Input image:



Output image:



Experiment 4: Perform Image Dilation

Aim

To perform the dilation operation on the given image using OpenCV.

Algorithm

1. Start the program.
2. Import the OpenCV and NumPy libraries.
3. Read the input image from the specified path.

4. Create a structuring element (kernel) of size 5×5 .
5. Apply the dilation operation using `cv2.dilate()`.
6. Increase the bright regions of the image.
7. Store the processed image.
8. Display the original image.
9. Display the dilated image.
10. Wait for a key press and close all windows.

Python Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\admin\Downloads\image.jfif")

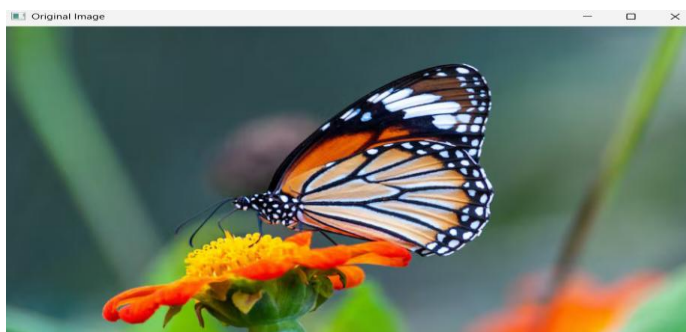
kernel = np.ones((5,5),np.uint8)

dilate = cv2.dilate(img,kernel,iterations=1)

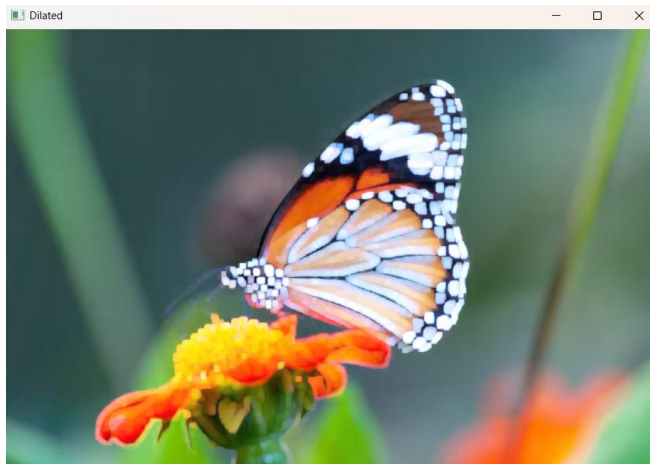
cv2.imshow("Original",img)
cv2.imshow("Dilated",dilate)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input Image



Output Image



Result

Thus, the dilation operation was successfully performed on the given image

